scienceStack

Rigorous Scientific Engineering Workflows

Operational Manual for the Google Antigravity Plugin

Plugin Specification & Operational Guide

Version 0.22.0 — October 2026

Components:
12 Skills
6 Playbooks, 6 Principles
2 Subagents
4 Lifecycle Hooks
1 Directive Rule File

Domains:
PDEs, CFD, orbital mechanics
Stiff ODE kinetics
Numerical linear algebra

Adapted from the `pstack` architecture by Lauren Tan (poteto) at Cursor.
Specialized for equation provenance, dimensional analysis, and numerical verification in Google Antigravity.

Contents

Executive Summary ii

1 Installation and How to Use 1
- 1.1 Installing the Plugin . . . 1
- 1.2 Skills . . . 1
- 1.3 Runtime Directives . . . 2

2 Architecture 3
- 2.1 Subagents and Models . . . 3
- 2.2 Where Things Live . . . 3

3 The Six Principles 4

4 Playbooks 6

5 Safety Hooks and Tools 7
- 5.1 PreToolUse Safety Gate . . . 7
- 5.2 Building Both Sides of a Comparison . . . 7
- 5.3 Verification Hooks . . . 7
- 5.4 AST Auditor . . . 8
- 5.5 MMS and Jacobian Generators . . . 8
- 5.6 Reference Benchmarks . . . 8

6 Testing 9

Recommended Literature & References 10

Executive Summary

Generated code that satisfies local tests can still be wrong in the ways that matter to computational science: a flipped sign in a diffusion operator, an off-by-one in a stencil, or an explicit matrix inverse that squares the condition number. **scienceStack** packages a small set of enforceable rules, workflows, and tools for Google Antigravity that target exactly these failures. It is organized around six principles:

1. **Equation provenance**: every implemented formula carries its continuous LaTeX form, a literature citation, units, and sign conventions.
2. **Dimensional discipline**: both sides of every equation have the same dimensions; arguments of transcendental functions are dimensionless.
3. **Numerical stability first**: no explicit inversion, guarded denominators, and `log1p`/`expm1` for small arguments.
4. **Conservation laws**: Hamiltonian systems use symplectic integrators; energy, momentum, and angular momentum are tracked in tests.
5. **Physics-modular architecture**: pure physical laws live in stateless modules, decoupled from time steppers, meshes, and I/O.
6. **Analytical limits and manufactured solutions**: solvers are verified against closed-form solutions or the Method of Manufactured Solutions with explicit error norms.

Chapter 1

Installation and How to Use

1.1 Installing the Plugin

The plugin complies with Agent Plugins 1.0.0; its root manifest is `plugin.json`.

```
# 1. Validate compliance with Agent Plugins 1.0.0 (Strict mode)
agy plugin validate /home/seo3919/AI_skills/scienceStack_antigravity
   --strict

# 2. Install globally in Antigravity
agy plugin install /home/seo3919/AI_skills/scienceStack_antigravity

# Or workspace-local:
# Place scienceStack_antigravity in .agents/plugins/sciencestack
```

Verify installation status:

```
agy plugin list
# - sciencestack (enabled, 0.22.0)
#   Components: 12 skills, 2 agents, 1 rule, 4 hooks
```

1.2 Skills

Skills can be triggered via interactive slash commands or dynamically by Antigravity agents:

- `/science-mode <task>`: master workflow. Selects exactly one of six playbooks and enforces the six principles. User-invoked.
- `/setup-sciencestack`: writes per-role model choices to configuration. User-invoked.
- `astronomy-computing`: Kepler solvers, symplectic N-body dynamics, SO(3) coordinate frames.
- `accelerated-scientific-computing`: memory layout, SIMD vectorization, and precision guards.
- `check-against-reference`: runs the user's solver on Kepler's equation, Lane-Emden, the Sod shock tube, or Robertson kinetics, or compares its output with golden data, through `scripts/check_against_reference.py`. The script decides (exit status 0 pass, 1 fail, 2 nothing checked, 3 verdict withheld or manifest weakened); `convergence-ode` and `convergence-bvp` measure observed order of convergence; `-repeat` detects nondeterminism; every run appends to a SHA-256 bound evidence log (`.verification/log.jsonl`); tolerance loosening after failure on unchanged files is withheld (exit 3) unless justified; `run-manifest` re-runs committed manifests.

- `run-tests`: runs unittest or pytest through `scripts/run_tests.py`, extracts exact counts from framework internals, fails when nothing ran or everything was skipped, and records results bound to the worktree hash; `run_tests.py fresh` confirms whether tests passed on current files.
- `test-ratchet`: `scripts/test_ratchet.py -baseline-ref REF` parses ASTs of test files against a git baseline and exits 3 on unapproved looser tolerance, removed tests, fewer assertions, new skip/xfail, or fewer parametrize cases; approvals go in `test-ratchet-approvals.json`.
- `numerical-stability-audit`: audits explicit matrix inversion, cancellation, and unseeded random number generator states via `scripts/audit_numerical_ast.py`.
- `precision-audit`, `scientific-equation-verifier`, `golden-dataset-benchmark`, `benchmark-checkli` audit and verification skills loaded on demand.

1.3 Runtime Directives

Directives are defined in `rules/AGENTS.md`: the core directives (provenance, stability, symplectic integration, modularity, reference checks, test ratchet, evidence logs) and absolute script locations.

Chapter 2

Architecture

2.1 Subagents and Models

Agent	Model	Tools	Role
`science-master`	`flash` / `inherit`	all	kernels, MMS tests, refactors, benchmark runners
`theory-reviewer`	`pro`	read-only	line-by-line audit of equations, signs, index order, invariants

Table 2.1: Subagents. `theory-reviewer` operates in a read-only context.

2.2 Where Things Live

```
plugin.json                          Agent Plugins 1.0.0 manifest
hooks/hooks.json                     PreToolUse, PostToolUse, and StopGate
    hooks
rules/AGENTS.md                      Antigravity runtime directives
agents/                              science-master, theory-reviewer
references/                          compact delegate principles (read by
    science-master)
skills/science-mode/                 playbooks/ (6) and principles/ (6)
skills/                              check-against-reference, run-tests,
    test-ratchet, ...
scripts/                             hooks, AST auditor, MMS and Jacobian
    generators
benchmarks/                          analytic reference solvers
tests/                               unit and integration tests
```

Chapter 3

The Six Principles

Scientific Principle: Equation Provenance

Document the governing equation in the docstring, for example

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u \quad \text{[SI]},$$

with its literature source, coordinate system, and sign convention. The AST auditor reports scientific functions whose docstring has no LaTeX equation.

Scientific Principle: Dimensional Discipline

For every equation [LHS] = [RHS]. Arguments of exp, ln, sin must be dimensionless. Non-dimensionalize coupled systems with characteristic scales.

Scientific Principle: Numerical Stability First

For a linear system $A\mathbf{x} = \mathbf{b}$ the relative error obeys

$$\frac{\|\delta \mathbf{x}\|}{\|\mathbf{x}\|} \lesssim \kappa(A) \left(\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta \mathbf{b}\|}{\|\mathbf{b}\|} \right).$$

Use `solve(A, b)` or a factorization rather than forming A^{-1}. Replace $\ln(1+x)$ by `log1p`, $e^x - 1$ by `expm1`, $1 - \cos x$ by $2\sin^2(x/2)$.

Scientific Principle: Conservation Laws

Hamiltonian systems $H(\mathbf{q}, \mathbf{p})$ use symplectic integrators (kick-drift-kick leapfrog, Yoshida splitting), which conserve a nearby shadow Hamiltonian $\tilde{H} = H + O(\tau^2)$ and show no secular energy drift. Track $E(t)$, $\mathbf{P}(t)$, $\mathbf{L}(t)$ in tests with an explicit tolerance. Explicit Runge–Kutta is not used for long-term orbit integration.

Scientific Principle: Physics-Modular Architecture

Forces, equations of state, and rate equations are pure, stateless functions in a `physics/` module that other solvers can import without a time stepper, mesh, or I/O.

Scientific Principle: Analytical Limits and MMS

For a differential operator $\mathcal{L}$, choose a smooth u_{mms}, compute $f = \mathcal{L}[u_{\text{mms}}]$ symbolically, solve $\mathcal{L}[u] = f$ numerically and check that the error norm falls with the expected order p under grid refinement:

$$p_{\text{obs}} = \frac{\ln(e_h / e_{h/2})}{\ln 2}.$$

Chapter 4

Playbooks

`/science-mode` selects and executes the specific playbook matching the task.

Playbook	Purpose
`equation-implementation`	Turn a published equation into verified code with LaTeX docstrings, dimension checks, and limit tests.
`numerical-verification`	Truncation-error order, grid-refinement convergence, conservation stability.
`perf-issue`	Profile and remove a bottleneck while asserting L_2 parity with the previous output.
`hillclimb`	Iterate against a quantitative target: one change, one measurement, keep or revert, with a decision log.
`bug-fix`	Reproduce on a minimal analytic case, find the root cause by evidence, failing test first.
`refactoring`	Pin behavior with a characterization test, then restructure into stateless physics modules.

Chapter 5

Safety Hooks and Tools

5.1 PreToolUse Safety Gate

`scripts/hook_pre_tool_guard.py` intercepts tool writes (`write_file`, `edit_file`, and terminal write invocations) and blocks proposed code that contains three critical hazards:

1. **Explicit matrix inversion**: `numpy.linalg.inv`, `scipy.linalg.inv`, `torch.linalg.inv`, `torch.inverse`, and JAX/CuPy/SciPy-sparse equivalents. Names are resolved through AST import aliases.
2. **Truncated π**: literal values `3.14159` and `3.1416`.
3. **Mutable NumPy default argument**: `def f(x=np.zeros(3))`.

Comments, docstrings, and strings are parsed away by the AST and never trigger the gate.

5.2 Building Both Sides of a Comparison

A manifest `compare` check can produce its reference and/or its test output by running programs instead of trusting committed files: `ref_from` and `test_from` give argument lists (never a raw shell), the output (`stdout` or file path), and `pinned`, the SHA-256 of every source and input file. A changed pinned file fails the check before anything runs. For `ref_from`, the committed reference must reproduce the rebuilt numbers within `reproduce_rtol` and `reproduce_atol`.

5.3 Verification Hooks

In a git project that uses scienceStack verification (contains `reference-checks.json`, `test-ratchet-approvals` or `.verification/`):

- `scripts/hook_test_guard.py` (PreToolUse): Compares proposed edits against baseline ASTs and denies/asks if a test is weakened (looser tolerance, removed test/assertion, new skip/xfail, fewer parametrizations).
- `scripts/hook_record_edit.py` (PostToolUse): Records files modified during the active session.
- `scripts/hook_stop_gate.py` (StopGate): Blocks completion once if edited solvers or compared outputs lack fresh passing checks, recorded tests are stale, or edited tests lack fresh ratchet runs.

Hooks fail closed: internal crashes or unexpected formats deny modifications until addressed. Set `SCIENCESTACK_HOOKS=off` in the environment to temporarily bypass verification hooks.

Numerical Safety Alert: Limits

The gate cannot see an inversion built dynamically (`getattr`), `np.matrix` objects used through `.I`, or code written by unhooked external processes. It is a safety net, not an absolute formal proof.

5.4 AST Auditor

`python3 scripts/audit_numerical_ast.py <file|dir> [-strict] [-json]` reports: `BAN_MATRIX_INV` (error); `CANCELLATION_LOG1P`, `CANCELLATION_EXPM1`, `CANCELLATION_ONE_MINUS_COS`, `PRECISION_FLOAT32`, `TRUNCATED_PI`, `MUTABLE_ARRAY_DEFAULT` (warnings); `MISSING_LATEX_PROVENANCE` (info).

5.5 MMS and Jacobian Generators

```
python3 scripts/mms_generator.py --exact "sin(pi*x)*exp(-t)" \
    --operator "diff(u, t) - alpha*diff(u, x, 2)" --params "alpha=0.1"
       --output fixture.py
python3 scripts/derive_jacobian.py --vars "x,y" --eqs "x**2 + y,
   sin(x)*y"
```

For $u = e^{-t}\sin \pi x$ and $\mathcal{L}[u] = u_t - \alpha u_{xx}$ the generator derives the forcing

$$f(x,t) = (\alpha\pi^2 - 1)\, e^{-t} \sin \pi x,$$

and writes a fixture whose convergence test is ready for solver attachment. The Jacobian generator emits `compute_jacobian(_state)` and analytic symbolic entries.

5.6 Reference Benchmarks

Script	Reference	Source
`cfd/exact_riemann.py`	Sod star state $p_* = 0.30313$, $u_* = 0.92745$ ($\gamma = 1.4$)	Toro (2009), Table 4.1
`astronomy/lane_emden_solver.py`	$\xi_1 = \pi$ for $n = 1$; 3.65375 for $n = 1.5$; 6.89685 for $n = 3$	standard polytrope values
`astronomy/kepler_solver.py`	conserved E and $\mathbf{L}$	analytic invariants
`astronomy/symplectic_nbody.py`	bounded energy error, conserved $\mathbf{P}$, $\mathbf{L}$	symplectic structure
`astronomy/coordinate_transforms.py`	$R^T R = I$, $\det R = 1$	SO(3)
`kinetics/robertson_reference.py`	$y_1 + y_2 + y_3 = 1$	mass conservation

Table 5.1: Benchmarks and what each is checked against.

Chapter 6

Testing

```
pytest tests/
```

The 594-test suite verifies safety hooks (import aliases, formatting resilience), the AST auditor, MMS and Jacobian generators, reference benchmarks, convergence order calculations, ratchet mechanics, and hash-chained verification evidence ledgers.

Recommended Literature & References

1. Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM.
2. Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations*, 4th ed. Johns Hopkins University Press.
3. Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM.
4. Hairer, E., Lubich, C., & Wanner, G. (2006). *Geometric Numerical Integration*, 2nd ed. Springer.
5. Roache, P. J. (2009). *Fundamentals of Verification and Validation*. Hermosa Publishers.
6. Salari, K., & Knupp, P. (2000). *Code Verification by the Method of Manufactured Solutions*. Sandia National Laboratories, SAND2000-1444.
7. Toro, E. F. (2009). *Riemann Solvers and Numerical Methods for Fluid Dynamics*, 3rd ed. Springer.
8. Hairer, E., & Wanner, G. (1996). *Solving Ordinary Differential Equations II*, 2nd ed. Springer.